



Saint Columban College
College of Computing Studies
Pagadian City



Transformation

Transformation 



Easy

Reverse Engineering

picoCTF 2021

AUTHOR: MADSTACKS

Description

I wonder what this really is...

```
enc".join([chr((ord(flag[i]) << 8) + ord(flag[i + 1])) for i in range(0, len(flag), 2))
```

Hints 

1

73,902 users solved



65% Liked



picoCTF{FLAG}

Submit Flag

Author: The Analyst: Hyposelenia

Challenge: transformation

Category: Reverse Engineering

Date: 04/04/26



I. Objective

The objective of this challenge is to understand how encoded text can be transformed at the byte level and how to reverse it.

The goal is to analyze the encoding method and **reconstruct the original message (flag)**.

II. Background

Computers store text using **bytes**.

- 1 byte = 8 bits
- Some encodings use **1 byte per character**
- Others use **2 bytes (16 bits)** per character

In this challenge, the text looked unusual because it was encoded using a **16-bit format**, where:

Each character actually contains TWO smaller characters hidden inside

III. Tool Used

Notepad - Used to view the encoded text

Online Python Interpreter (Programiz) - Used to write and execute decoding script

IV. Methodology

1. Downloaded the provided file

`enc ".jc`

2. Observed unusual characters: 瀰捌宏規ほ形梲獠楮獠ズツ摟潦弼
形寔坵捡似



灑捌宏規ㄣ形梲獍楮獐ㄣ搜潦弼形寔坵捡似

3. Recognized this as likely **UTF-16 / 16-bit encoded data**
4. Created a Python script to extract hidden bytes

```
1 encoded = "灑捌宏規ㄣ形梲獍楮獐ㄣ搜潦弼形寔坵捡似"
2
3 decoded = ""
4
5 for ch in encoded:
6     val = ord(ch)
7     decoded += chr(val >> 8)      # high byte
8     decoded += chr(val & 0xFF)   # low byte
9
10 print(decoded)
```

5. Ran the script and reconstructed the original text
6. Retrieved the **flag**

V. Result

picoCTF{16_bits_inst34d_of_8_b7f62ca5}

```
picoCTF{16_bits_inst34d_of_8_b7f62ca5}

=== Code Execution Successful ===
```



```
encoded = "豔捌畧覩ㄣ形梃獍楮獾𪛗樓潦彌形寔坵捡似"

decoded = ""

for ch in encoded:
    val = ord(ch)
    decoded += chr(val >> 8)  # high byte
    decoded += chr(val & 0xFF) # low byte

print(decoded)
```

VI. Explanation

You might/must be confused to what incarnation was that string? Well...

What kind of encoding is this?

This is NOT encryption but;

This is **encoding**

Difference?

- Encryption = secret + needs key
- Encoding = just a different format

What is happening here?

Each strange character like:

豔

Actually contains **2 hidden characters inside**

Because:

- It uses **16 bits (2 bytes)**
- Instead of normal **8 bits (1 byte)**



How do we decode it?

We split each character into:

- **High byte** (first 8 bits)
- **Low byte** (last 8 bits)

Let's Proceed to Line-by-Line Code Explanation

1. Encoded string

```
encoded = "灑倒罢覩诃形梯獐楮獠焱樓潦彌彥ㄥ一て尅"
```

This is the strange text we want to decode

Empty result

```
decoded = ""
```

We will build the final message here

Loop through each character

```
for ch in encoded:
```

Go through each weird character one by one

Convert character to number

```
val = ord(ch)
```

ord() means:

“Give me the number value of this character”

Example:

A → 65

Get the FIRST half (high byte)

```
decoded += chr(val >> 8)
```

>> 8 means:

“Shift bits to the right by 8”

💡 Meaning:

- Take the **first 8 bits**



- This becomes a character

Get the SECOND half (low byte)

decoded += chr(val & 0xFF)

& 0xFF means:

“Keep only the last 8 bits”

Meaning:

- Extract the **second character**

Print result

print(decoded)

Shows the decoded message (the flag)

How to Spot This Type of Encoding???

Signs:

1. Weird/unreadable characters:

滯捌宏規

2. Looks like **Asian characters but doesn't make sense**
3. Text length is shorter than expected

Key Idea:

“Something is packed together”

Type	Size	Meaning
8-bit	1 byte	normal text (ASCII)
16-bit	2 bytes	2 characters packed
32-bit	4 bytes	more complex encoding



Saint Columban College
College of Computing Studies
Pagadian City



VII. Conclusion

This challenge demonstrates how data can be transformed using different encoding formats.

By recognizing the structure of the encoded text and reversing the transformation, the original message was successfully recovered.

This reinforces an important concept in reverse engineering:

Understanding data representation is key to decoding hidden information.

— The Analyst: Hyposelenia